Reinforcement Learning

Notes

MDP theory · Dynamic Programming · Bandit problems
Monte Carlo · Temporal-Difference · SARSA · Q-Learning
Off-policy methods · Eligibility Traces · Function Approximation
Deep RL · Policy Gradient Methods

For Study Use

Contents

1 Fundamental Concepts **4**
1.1 What is Reinforcement Learning? . . . 4
1.2 Sequential Decision Making . . . 4
1.3 Agent–Environment Interaction . . . 4

2 Markov Decision Processes **5**
2.1 The Markov Property . . . 5
2.2 One-Step Dynamics . . . 5
2.3 Finite MDPs . . . 6
2.4 Example: The Recycling Robot . . . 6
2.5 The Return . . . 6

3 Episodic and Continuing Tasks **6**
3.1 Episodic Tasks . . . 6
3.2 Continuing Tasks . . . 7
3.3 Unified Notation and the Discount Factor . . . 7
3.4 Goal and the Reward Hypothesis . . . 7

4 Policy **7**
4.1 Deterministic Policy . . . 7
4.2 Stochastic Policy . . . 7
4.3 Policy Classifications . . . 8

5 Value Functions **8**
5.1 State-Value and Action-Value Functions . . . 8
5.2 The Bellman Expectation Equation . . . 8
5.3 Gridworld Example . . . 9

6 Comparing Policies and Optimality **9**
6.1 Partial Order and Optimal Policy . . . 9
6.2 Optimal Value Functions and Bellman Optimality . . . 9

7 Dynamic Programming **10**
7.1 Why Dynamic Programming? . . . 10
7.2 Iterative Policy Evaluation . . . 10
7.3 Policy Improvement . . . 11
7.4 Policy Iteration . . . 11

7.5 Value Iteration . . . 11
7.6 Asynchronous Dynamic Programming and Complexity . . . 12

8 The k-Armed Bandit Problem **12**
8.1 Problem Setup . . . 12
8.2 Estimating Action Values from Experience . . . 12
8.3 Non-Stationary Bandits and Constant Step Size . . . 13

9 Action Selection Strategies **13**
9.1 The Exploration–Exploitation Dilemma . . . 13
9.2 Greedy Strategy and ε-Greedy . . . 13
9.3 Optimistic Initial Values . . . 13
9.4 Upper Confidence Bound (UCB) . . . 14

10 Monte Carlo Methods **14**
10.1 Motivation: Beyond Dynamic Programming . . . 14
10.2 Monte Carlo Policy Evaluation . . . 14
10.2.1 Worked Example . . . 15
10.3 Blackjack: A Concrete Application . . . 15
10.4 MC Policy Iteration: Moving to Control . . . 15
10.5 ε-Soft Monte Carlo Policy Iteration . . . 16

11 Off-Policy Learning **16**
11.1 On-Policy vs. Off-Policy . . . 17
11.2 Importance Sampling . . . 17

12 Temporal-Difference Learning **17**
12.1 Motivation . . . 17
12.2 TD(0): Core Idea and Update . . . 18
12.3 Random Walk Example . . . 18
12.4 MC vs. TD: Bias-Variance Trade-off . . . 18
12.5 Bootstrapping and Sampling: Taxonomy . . . 19

13 SARSA and Q-Learning **19**
13.1 From Evaluation to Control . . . 19
13.2 SARSA: On-Policy TD Control . . . 19
13.3 Q-Learning: Off-Policy TD Control . . . 19
13.4 Bellman Equation Correspondence . . . 20
13.5 Cliff Walking: A Key Illustration . . . 20

14 n-Step Methods and Eligibility Traces **21**

14.1 n-Step Returns . . . 21

14.2 The λ-Return . . . 21

14.3 Eligibility Traces: Efficient Implementation of TD(λ) . . . 21

15 Function Approximation **22**

15.1 Why Function Approximation? . . . 22

15.2 Linear Function Approximation . . . 22

15.3 Semi-Gradient TD(0) . . . 22

15.4 Non-Linear Function Approximation and Neural Networks . . . 22

16 Deep Reinforcement Learning **23**

16.1 Motivation . . . 23

16.2 Deep Q-Network (DQN) . . . 23

16.3 DQN in Practice: Atari Games . . . 24

16.4 Limitations of DQN and Extensions . . . 24

17 Policy Gradient Methods **25**

17.1 Why Policy Gradients? . . . 25

17.2 The Policy Gradient Objective . . . 25

17.3 The Policy Gradient Theorem . . . 25

17.4 REINFORCE: Monte Carlo Policy Gradient . . . 26

17.5 Variance Reduction: The Baseline . . . 26

17.6 Actor-Critic Methods . . . 26

17.7 Comparing Value-Based and Policy-Based Methods . . . 27

18 Big Picture Summary **27**

1 Fundamental Concepts

1.1 What is Reinforcement Learning?

Reinforcement Learning (RL) is a branch of machine learning concerned with how an agent can learn to make good decisions by interacting with an environment. Unlike supervised learning, the agent is never told what the correct action is. Instead, it receives a scalar **reward** signal after each action, which tells it how well it did. The agent must figure out on its own — through trial and error — which actions lead to the best long-term outcomes.

This sets RL apart from the other main paradigms. In supervised learning, the model is trained on labelled pairs (x, y): for every input x, the correct output y is provided, and learning consists of minimising a prediction error. In reinforcement learning, no correct output is ever given. The agent can only evaluate how good its actions were *after the fact*, and it must reason about long-term consequences rather than just matching an immediate target.

Supervised Learning vs. Reinforcement Learning

	Supervised Learning	**Reinforcement Learning**
Training signal	Labelled pairs (x, y)	Scalar reward $R \in \mathbb{R}$
Feedback type	Correct output provided	Evaluative only
Goal	Learn $f(x) \approx y$	Maximise cumulative reward

1.2 Sequential Decision Making

A defining feature of RL is that the agent does not make a single isolated decision, but a *sequence* of decisions $A_0, A_1, A_2, \ldots$ over time. Each action does more than produce an immediate reward: it also determines which state the agent enters next, and thus which options and rewards are available in the future. The quality of a decision must therefore be judged not just by its immediate effect, but by its entire chain of future consequences.

A simple example makes this clear. Suppose the agent can choose between:

$$A_1 : +10 \rightarrow -100 \quad \Rightarrow \quad \text{total} = -90 \qquad\qquad A_2 : +3 \quad \Rightarrow \quad \text{total} = +3$$

A greedy agent would always pick A_1 because $10 > 3$, but the rational choice is A_2. Maximising immediate reward is not sufficient — one must think ahead.

Other key properties of sequential decision making:

- The optimal action depends on the current context (state).
- The agent is never told which actions are correct; it must discover this through interaction.
- Actions may have delayed effects: a choice made now can influence rewards many steps later.
- The objective is long-term performance, not short-term gain.

1.3 Agent–Environment Interaction

The interaction is modelled in discrete time $t = 0, 1, 2, \ldots$ At each step:

Variables at Each Time Step

- $S_t \in \mathcal{S}$: the **state** at time t, representing the agent's current situation. $\mathcal{S}$ is the set of all possible states.
- $A_t \in \mathcal{A}(S_t)$: the **action** chosen at time t. $\mathcal{A}(S_t)$ is the set of available actions in state S_t.
- $R_{t+1} \in \mathbb{R}$: the **reward** received after taking A_t. Note the index shift: reward arrives one step after the action.
- $S_{t+1} \in \mathcal{S}$: the **next state** the agent transitions into.

The cycle unfolds as: $S_t \to A_t \to (R_{t+1}, S_{t+1}) \to A_{t+1} \to \cdots$

The full sequence of interactions is called a **trajectory**: $S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots$

Objective

The agent's goal is to maximise the expected cumulative reward: max $\mathbb{E}[\sum_{t=0}^{\infty} R_t]$. The expectation accounts for stochasticity in the environment and/or the policy.

2 Markov Decision Processes

2.1 The Markov Property

To make the RL problem tractable, we rely on the **Markov property**: the future depends only on the current state and action, not on the full history of past states and actions. In other words, the current state contains all the information that is relevant for predicting what happens next.

Markov Property

$$p(s', r \mid s_0, a_0, \ldots, s_t, a_t) = p(s', r \mid s_t, a_t)$$

The next state s' and reward r depend only on the current state s_t and action a_t.

This is not a restrictive assumption in practice. If the raw observations do not satisfy the Markov property (e.g., a robot that can only see what is directly in front of it), the state representation can be augmented to include enough historical information. For example, using the last two frames of a video game as the state restores the Markov property.

2.2 One-Step Dynamics

A system satisfying the Markov property is called a **Markov Decision Process** (MDP). Its dynamics are fully captured by:

One-Step Dynamics

$$p(s', r \mid s, a) = \Pr\{S_{t+1} = s',\ R_{t+1} = r \mid S_t = s,\ A_t = a\}$$

This function assigns a probability to every possible next state and reward, given the current state and action. It must satisfy $\sum_{s'} \sum_r p(s', r \mid s, a) = 1$ for all (s, a).

From $p(s', r \mid s, a)$, two useful quantities can be derived by marginalisation:

Transition probability (probability of reaching s', regardless of reward):

$$p(s' \mid s, a) = \sum_r p(s', r \mid s, a)$$

Expected reward (average reward for taking action a in state s):

$$r(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_r r \sum_{s'} p(s', r \mid s, a)$$

2.3 Finite MDPs

When $\mathcal{S}$ and $\mathcal{A}$ are finite sets and the Markov property holds, the problem is a **finite MDP**, fully specified by $(\mathcal{S},\ \mathcal{A}(s),\ p(s', r \mid s, a))$.

2.4 Example: The Recycling Robot

A mobile robot collects cans and must decide how to manage its battery. States are $\mathcal{S} = \{high,\ low\}$ (energy level). Available actions depend on the state: when the battery is high, the robot can search or wait; when it is low, it can also recharge: $\mathcal{A}(high) = \{search,\ wait\}$, $\mathcal{A}(low) = \{search,\ wait,\ recharge\}$.

Rewards equal the number of cans collected. Searching is more rewarding than waiting ($r_{\text{search}} > r_{\text{wait}} > 0$) but consumes more energy. If the robot searches while the battery is low, it risks running out of power and incurs a penalty. The transition probabilities are parameterised by α (probability of staying at high energy after searching) and β (probability of staying at low energy after searching), which together define the full dynamics $p(s', r \mid s, a)$.

2.5 The Return

The agent cares about total accumulated reward, not just the next step. This is formalised as the **return** G_t.

Return G_t

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

where $\gamma \in [0, 1)$ is the **discount factor**. A reward received k steps in the future is worth γ^k times its face value. Three common choices: total reward ($\gamma = 1$, episodic only), discounted reward ($\gamma < 1$, ensures convergence), average reward (used in some continuing settings).

3 Episodic and Continuing Tasks

3.1 Episodic Tasks

In **episodic tasks**, the interaction divides naturally into finite, independent sequences called **episodes**. Each episode starts in some initial state and ends when the agent reaches a **terminal state**, after which the environment resets. Examples include game matches, navigation trips, and board games.

An episode is a trajectory $S_0, A_0, R_1, S_1, \ldots, S_T$, where T is the final time step and S_T is the terminal state. The return is the total reward until termination: $G_t = R_{t+1} + R_{t+2} + \cdots + R_T$.

3.2 Continuing Tasks

In **continuing tasks**, the interaction never ends. There is no terminal state; the process runs indefinitely. Examples include process control, trading, and autonomous robots.

Without discounting, the sum $\sum_{t=0}^{\infty} R_t$ could diverge. Discounting ensures a finite, well-defined return: $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$, $\gamma \in (0, 1)$. If rewards are bounded by $|R_t| \leq R_{\max}$, then $G_t \leq R_{\max}/(1-\gamma) < \infty$.

3.3 Unified Notation and the Discount Factor

Both settings are unified by $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$. For episodic tasks, terminal states are modelled as **absorbing states** that loop to themselves with zero reward.

The discount factor γ has a direct interpretation: $\gamma = 0$ makes the agent completely myopic (only immediate reward matters); $\gamma \to 1$ makes the agent far-sighted (future rewards count nearly as much as present ones); $\gamma = 1$ is only valid in episodic settings.

3.4 Goal and the Reward Hypothesis

A goal specifies *what* the agent should achieve, not *how*. The **reward hypothesis** is the central claim:

Reward Hypothesis

All goals and purposes of an intelligent agent can be expressed as the maximisation of the expected value of the cumulative sum of a scalar reward signal.

Common reward designs: $R = +1$ when the goal is reached and 0 otherwise (goal-based); or $R = -1$ at every non-terminal step and 0 at the goal (step-penalty, which incentivises reaching the goal quickly). Designing rewards well is non-trivial: a poorly specified reward can lead the agent to find unintended loopholes.

4 Policy

A **policy** π is the agent's strategy: it specifies how the agent selects actions given the current state. The policy is the central object in RL — finding a good policy is the ultimate goal.

4.1 Deterministic Policy

The simplest type maps each state to a single action: $\pi : \mathcal{S} \to \mathcal{A}$, $\pi(s) = a$. The behaviour is fully determined and contains no randomness. Such a policy can be represented as a lookup table.

4.2 Stochastic Policy

In the more general case, a policy assigns a probability distribution over actions for each state:

$$\pi(a \mid s) = \Pr\{A_t = a \mid S_t = s\}, \qquad \sum_a \pi(a \mid s) = 1, \quad \pi(a \mid s) \geq 0$$

A deterministic policy is the special case where one action has probability 1 and all others have probability 0. Stochastic policies are important because they allow natural exploration and arise in game theory and partial observability settings.

4.3 Policy Classifications

Type	Definition	Example
Markovian	Depends only on current s	$\pi(a \mid s)$
Non-Markovian	Depends on full history	$\pi(a \mid s_0, a_0, \ldots, s_t)$
Stationary	Does not depend on time t	$\pi_t = \pi \ \forall t$
Non-stationary	Explicitly depends on t	$\pi_t(a \mid s)$

A non-Markovian policy can often be made Markovian by augmenting the state (e.g., including previous actions). Most RL theory focuses on **stationary Markovian** policies, which are sufficient for finite MDPs.

5 Value Functions

5.1 State-Value and Action-Value Functions

Value functions capture how good it is to be in a state (or to take an action) when following a given policy. They are the key tool for evaluating and improving policies.

State-Value Function $V_\pi(s)$

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \,\middle|\, S_t = s\right]$$

This is the expected cumulative discounted reward when starting from s and following π thereafter. A high value means the state is promising under π.

Action-Value Function $Q_\pi(s, a)$

$$Q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s,\ A_t = a]$$

Expected return starting from s, taking action a first, then following π. The difference from V_π is that the first action is fixed. This makes Q_π useful for comparing actions: $Q_\pi(s, a_1) > Q_\pi(s, a_2)$ means a_1 is better than a_2 in state s under π.

Probabilistic interpretation. When rewards are binary (win $= +1$, lose $= 0$), $V_\pi(s)$ equals the probability of winning from s under π.

5.2 The Bellman Expectation Equation

Value functions satisfy a fundamental recursive identity: the value of a state equals the expected immediate reward plus the discounted value of the next state.

Bellman Expectation Equation

$$V_\pi(s) = \sum_a \pi(a \mid s) \left(r(s, a) + \gamma \sum_{s'} p(s' \mid s, a)\, V_\pi(s') \right)$$

The outer sum averages over actions chosen by π; the inner sum averages over stochastic transitions. Together they give the full expected future return from s under π.

For the action-value function:

$$Q_\pi(s,a) = r(s,a) + \gamma \sum_{s'} p(s' \mid s,a) \sum_{a'} \pi(a' \mid s')\, Q_\pi(s',a')$$

The Bellman equation transforms the problem of computing a value function into *solving a linear system*: for a finite MDP with $|\mathcal{S}|$ states, it provides $|\mathcal{S}|$ linear equations in $|\mathcal{S}|$ unknowns. Exact solution is feasible for small problems but computationally prohibitive for large state spaces, motivating approximate and learning-based methods.

5.3 Gridworld Example

Consider a gridworld with states $\{A, B, C, D\}$, a uniform stochastic policy $\pi(a \mid s) = \frac{1}{4}$, and discount factor $\gamma = 0.7$. The Bellman equation for state A is:

$$V_\pi(A) = \frac{1}{4} \sum_{a \in \mathcal{A}(A)} \big(r(A,a) + \gamma\, V_\pi(s'(A,a))\big)$$

Repeating for all states gives a 4×4 linear system whose solution is V_π. Each equation says: the value of a state equals the average (over the four equiprobable actions) of the immediate reward plus the discounted value of the state I land in.

6 Comparing Policies and Optimality

6.1 Partial Order and Optimal Policy

We compare policies via their value functions: $\pi \geq \pi'$ iff $V_\pi(s) \geq V_{\pi'}(s)$ for all s. This is a partial order because two policies are not always comparable (one might be better in some states, worse in others).

Optimal Policy π^*

π^* is optimal if $V_{\pi^*}(s) \geq V_\pi(s)$ for all $s \in \mathcal{S}$ and all π. A fundamental theorem guarantees that for any finite MDP, at least one optimal *deterministic* stationary policy always exists. This is non-trivial: among all stochastic and history-dependent policies, a deterministic one always suffices.

6.2 Optimal Value Functions and Bellman Optimality

$$V^*(s) = \max_\pi V_\pi(s) \qquad Q^*(s,a) = \max_\pi Q_\pi(s,a)$$

Unlike the Bellman expectation equation (linear), the optimal functions satisfy *non-linear* equations due to the max:

Bellman Optimality Equations

$$V^*(s) = \max_a \left(r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, V^*(s') \right)$$

$$Q^*(s,a) = r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\ \max_{a'} Q^*(s',a')$$

Once V^* or Q^* is known, the optimal policy is recovered by a greedy step:

$$\pi^*(s) = \arg\max_a \left(r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, V^*(s') \right) = \arg\max_a Q^*(s,a)$$

Using Q^* is particularly convenient: $\pi^*(s) = \arg\max_a Q^*(s,a)$ requires no explicit knowledge of r or p.

A finite MDP with $|\mathcal{S}|$ states and $|\mathcal{A}|$ actions has $|\mathcal{A}|^{|\mathcal{S}|}$ deterministic policies — an exponentially large space. Direct enumeration is completely infeasible. The key insight is to *compute* optimal value functions instead, and extract π^* from them in one greedy step.

7 Dynamic Programming

7.1 Why Dynamic Programming?

Dynamic Programming (DP) exploits the recursive Bellman structure to compute optimal policies efficiently. It requires a complete model of the environment (p and r must be known). The approach decomposes into two alternating steps: **policy evaluation** (compute V_π for a fixed π) and **policy improvement** (derive a better policy from V_π).

7.2 Iterative Policy Evaluation

The Bellman equation for V_π is a linear fixed-point equation. We solve it iteratively by starting from an arbitrary V_0 and repeatedly applying the Bellman update. Each complete pass over all states is called a **sweep**:

$$V_{k+1}(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s,a) \big[r + \gamma V_k(s') \big]$$

The sequence $V_0 \to V_1 \to \cdots$ converges to V_π regardless of the initialisation.

Iterative Policy Evaluation

Input: policy π, threshold $\theta > 0$

Initialise $V(s) \leftarrow 0\ \forall s$ (terminal states fixed at 0)
repeat
  $\Delta \leftarrow 0$
  for each $s \in \mathcal{S}$ **do**
    $v \leftarrow V(s)$
    $V(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s,a) \big[r + \gamma V(s') \big]$
    $\Delta \leftarrow \max(\Delta,\ |v - V(s)|)$
  end for

9: **until** $\Delta < \theta$

In-place updates: the new $V(s)$ is used immediately within the same sweep, which accelerates convergence compared to keeping old and new values separate.

Gridworld example. With $\gamma = 1$, reward -1 at every step, and a uniform policy, $V_0 = 0$ everywhere. After the first sweep $V_1 = -1$ for all non-terminal states. As sweeps continue, states farther from terminal states get increasingly negative values, reflecting the greater number of steps needed to reach termination.

7.3 Policy Improvement

Given V_π, we improve the policy by acting greedily: in each state, select the action that maximises expected return according to the current value estimate.

$$\pi'(s) = \arg\max_a \left(r(s,a) + \gamma \sum_{s'} p(s' \mid s,a)\, V_\pi(s') \right) = \arg\max_a Q_\pi(s,a)$$

Policy Improvement Theorem

Let π' be the greedy policy with respect to V_π. Then $V_{\pi'}(s) \geq V_\pi(s)$ for all s.

Proof sketch: $V_\pi(s) \leq Q_\pi(s, \pi'(s)) = \mathbb{E}_{\pi'}[R_{t+1} + \gamma V_\pi(S_{t+1}) \mid S_t = s] \leq \cdots \leq V_{\pi'}(s)$.
The first inequality holds because π' is greedy. The recursive unrolling gives the final bound. If $\pi' = \pi$, then π already satisfies the Bellman optimality condition and is optimal.

7.4 Policy Iteration

Alternating evaluation and improvement generates a monotonically improving sequence converging to π^*:

$$\pi_0 \xrightarrow{\text{eval}} V_{\pi_0} \xrightarrow{\text{improve}} \pi_1 \xrightarrow{\text{eval}} V_{\pi_1} \xrightarrow{\text{improve}} \cdots \to \pi^*$$

7.5 Value Iteration

Instead of fully evaluating each policy, perform one Bellman update per state and immediately apply the greedy step. Since the greedy policy is deterministic, the two steps collapse into one:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s,a) \big[r + \gamma V_k(s') \big]$$

This applies the Bellman optimality operator iteratively. $V_k \to V^*$ from any starting point.

Value Iteration

Input: threshold $\theta > 0$

1: Initialise $V(s) \leftarrow 0\ \forall s$ (terminal states fixed at 0)
2: **repeat**
3: $\quad \Delta \leftarrow 0$
4: $\quad$ **for** each $s \in \mathcal{S}$ **do**
5: $\quad\quad v \leftarrow V(s); \quad V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s,a) \big[r + \gamma V(s') \big]$
6: $\quad\quad \Delta \leftarrow \max(\Delta,\ |v - V(s)|)$

```
    end for
until Δ < θ
```

Output: $\pi^*(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma V(s')\big]$

Generalised Policy Iteration (GPI)

Both Policy Iteration and Value Iteration are instances of **GPI**: evaluation pushes V toward consistency with π; improvement pushes π toward being greedy w.r.t. V. In Policy Iteration these steps are complete and alternating. In Value Iteration they are interleaved at the finest granularity. Both converge; the trade-off is in the cost per iteration vs. number of iterations needed.

7.6 Asynchronous Dynamic Programming and Complexity

Standard DP sweeps all $|\mathcal{S}|$ states per iteration. **Asynchronous DP** relaxes this: states are updated one at a time, in any order. As long as every state is eventually updated infinitely often, convergence to V^* is guaranteed. This gives flexibility to focus computation on the most important states.

Value Iteration costs $\mathcal{O}(|\mathcal{S}|^2|\mathcal{A}|)$ per sweep — polynomial, but $|\mathcal{S}|$ often grows exponentially with the number of state variables (**curse of dimensionality**). Classical DP handles up to $\sim 10^6$ states; beyond that, approximate methods are necessary.

8 The k-Armed Bandit Problem

8.1 Problem Setup

The **k-armed bandit** strips away state: the agent repeatedly chooses one of k actions and receives a reward from an unknown distribution associated with that action. There are no transitions and the situation does not change between steps. This isolates the exploration–exploitation dilemma in its purest form.

True Action Value

$$q^*(a) = \mathbb{E}[R_t \mid A_t = a] = \sum_r p(r \mid a)\, r$$

The optimal action is $a^* = \arg\max_a q^*(a)$, unknown to the agent.

Example. An action with reward -11 or $+9$ each with probability 0.5 has $q^*(a) = -1$.

Clinical trial interpretation. Each action is a treatment; reward is 1 if successful, 0 otherwise. $q^*(a)$ is the unknown success probability. The agent must discover which treatment works best while minimising the number of patients on inferior treatments — a direct exploration–exploitation trade-off with real ethical stakes.

8.2 Estimating Action Values from Experience

Since $q^*(a)$ is unknown, we estimate it with the empirical average:

$$Q_t(a) = \frac{\sum_{i=1}^{t-1} R_i\, \mathbf{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbf{1}_{A_i=a}}$$

By the law of large numbers, $Q_t(a) \to q^*(a)$ as a is selected infinitely often.

Incremental form (avoids storing all past rewards):

$$Q_{n+1} = Q_n + \frac{1}{n}(R_n - Q_n)$$

New estimate = old estimate + step size × prediction error $(R_n - Q_n)$.

8.3 Non-Stationary Bandits and Constant Step Size

When reward distributions *change over time*, old observations become unreliable. A constant step size $\alpha \in (0, 1]$ adapts continuously:

$$Q_{n+1} = Q_n + \alpha(R_n - Q_n)$$

Expanding the recursion reveals this is an **exponential moving average**: $Q_{n+1} = \alpha R_n + (1 - \alpha)\alpha R_{n-1} + \cdots$ — recent observations receive higher weight, older ones decay exponentially. This allows the agent to track changing reward distributions.

9 Action Selection Strategies

9.1 The Exploration–Exploitation Dilemma

The fundamental tension: **exploitation** means selecting the action that currently appears best, maximising reward given current estimates; **exploration** means trying actions that may not look optimal right now, in order to gather more information and potentially find better options. These objectives conflict, and no universal solution exists.

9.2 Greedy Strategy and ε-Greedy

The pure greedy strategy $a_g = \arg\max_a Q_t(a)$ exploits fully but never explores. If estimates are imperfect early on, the agent may lock onto a suboptimal action permanently.

ε-Greedy Action Selection

$$A_t = \begin{cases} \arg\max_a Q_t(a) & \text{with probability } 1 - \varepsilon \\ \text{uniformly random action} & \text{with probability } \varepsilon \end{cases}$$

Every action is tried with positive probability at every step, ensuring all estimates eventually converge. Empirically: $\varepsilon = 0$ often converges to a suboptimal action; $\varepsilon = 0.01$ is slow to correct mistakes but performs well long-term; $\varepsilon = 0.1$ identifies the optimal action faster but wastes some steps on known-suboptimal choices.

9.3 Optimistic Initial Values

Initialise all estimates with an optimistically large value (e.g. $Q_1(a) = 2$ when true values are near 0–1). Each time an action is tried, its estimate drops toward the true value and becomes less attractive than the yet-untried actions. This drives the agent to try all actions *without explicit randomness* — exploration is induced entirely by the initial bias. The limitation is that exploration is temporary: once estimates converge, no further exploration occurs, making this unsuitable for non-stationary problems.

9.4 Upper Confidence Bound (UCB)

ε-greedy explores uniformly, ignoring uncertainty. UCB is smarter: it prefers actions that are either estimated to be good *or* are highly uncertain (rarely tried).

UCB Action Selection

$$A_t = \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right]$$

$Q_t(a)$ is the exploitation term; $c\sqrt{\ln t / N_t(a)}$ is the exploration bonus. When $N_t(a)$ is small (action rarely tried), the bonus is large and the action is preferentially selected. As $N_t(a)$ grows, the bonus shrinks. The $\ln t$ factor grows slowly with time, ensuring all actions keep receiving occasional visits even late in learning. UCB outperforms ε-greedy because it directs exploration toward actions where information is most needed.

10 Monte Carlo Methods

10.1 Motivation: Beyond Dynamic Programming

DP requires knowing $r(s, a)$ and $p(s' \mid s, a)$ exactly. In most real problems this model is unavailable. **Monte Carlo** (MC) methods replace analytical expectations with empirical averages over sampled episodes. No model is needed; learning proceeds purely from experience.

10.2 Monte Carlo Policy Evaluation

The value $V_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ is estimated by averaging observed returns from state s:

$$V_\pi(s) \approx \frac{1}{N(s)} \sum_{i=1}^{N(s)} G^{(i)}(s)$$

- **First-visit MC:** only the first occurrence of s per episode contributes.
- **Every-visit MC:** all occurrences contribute.

Both converge to $V_\pi(s)$ as episodes $\to \infty$. MC requires complete episodes because G_t can only be computed once the episode terminates.

Incremental update (avoids storing all returns): $V(s) \leftarrow V(s) + \alpha(G - V(s))$.

First-Visit MC Policy Evaluation

Input: policy π

```
Initialise V(s) arbitrarily; Returns(s) ← ∅ ∀s
loop for each episode:
    Generate episode following π: S_0, A_0, R_1, S_1, ..., S_T
    G ← 0
    for t = T − 1, ..., 0 do
        G ← γG + R_{t+1}
        if S_t ∉ {S_0, ..., S_{t−1}} then                    ▷ first visit
            Append G to Returns(S_t);   V(S_t) ← average(Returns(S_t))
        end if
    end for
```

11: **end loop**

Why process backwards? The return satisfies $G_t = R_{t+1} + \gamma G_{t+1}$. Processing from the end of the episode lets us compute all G_t values in a single backward pass.

Worked Example

Episode 1: $A \to B \to A \to$ Terminal, rewards $R_1 = 0,\ R_2 = 1,\ R_3 = 2,\ \gamma = 1$.

Backwards: $G_2 = 2,\ G_1 = 1 + 2 = 3,\ G_0 = 0 + 3 = 3$. First visits: $V(A) = 3,\ V(B) = 3$.

Episode 2: $A \to B \to$ Terminal, $R_1 = 1,\ R_2 = 0$. Returns: $G_1 = 0,\ G_0 = 1$.

After averaging: $V(A) = \frac{3+1}{2} = 2, \quad V(B) = \frac{3+0}{2} = 1.5$.

10.3 Blackjack: A Concrete Application

Blackjack is a natural episodic task for Monte Carlo, since the rules prevent a straightforward model.

State: the player's current sum (12–21), the dealer's one visible card, and whether the player holds a usable ace. This gives a state space of 200 states.

Actions: *hit* (draw another card) or *stick* (stop).

Rewards: $+1$ if the player wins, 0 for a draw, -1 for a loss.

Consider the fixed policy: stick if the sum is 20 or 21, otherwise hit. We simulate many games under this policy and, each time the agent visits a state, record the return (win/loss outcome). By averaging these returns over all visits to each state, we build up an estimate of V_π.

The result is a surface over the two-dimensional state space (player sum $\times$ dealer card), with separate surfaces for states with and without a usable ace. After 10 000 episodes, the surface is noisy but recognisable. After 500 000 episodes, it has converged to a smooth estimate of the true expected returns. States where the player has a high sum and the dealer shows a low card have high value (likely to win); states where the player must hit on a low sum against a dealer showing a high card have low value (likely to lose).

This example illustrates both the power and the limitation of MC: it can handle complex, model-free settings, but it needs many episodes to produce accurate estimates.

10.4 MC Policy Iteration: Moving to Control

For control (finding the optimal policy), we estimate $Q_\pi(s, a)$ rather than $V_\pi(s)$, since Q_π enables model-free policy improvement:

$$\pi'(s) = \arg\max_a Q_\pi(s, a), \qquad Q_\pi(s, a) \approx \frac{1}{N(s, a)} \sum_{i=1}^{N(s,a)} G^{(i)}(s, a)$$

For correct estimation, every (s, a) pair must be visited infinitely often. **Exploring starts** guarantee this: each episode begins at a randomly chosen (S_0, A_0).

MC Policy Iteration with Exploring Starts

1: Initialise $\pi(s)$, $Q(s, a)$ arbitrarily; Returns$(s, a) \leftarrow \emptyset$
2: **loop for each episode:**
3: Choose (S_0, A_0) uniformly at random (exploring starts)

```
    Generate episode S_0, A_0, R_1, ..., S_T following π
    G ← 0
    for t = T − 1, ..., 0 do
        G ← γG + R_{t+1}
        if (S_t, A_t) is first occurrence then
            Append G to Returns(S_t, A_t);   Q(S_t, A_t) ← average(...)
        end if
        π(S_t) ← arg max_a Q(S_t, a)
    end for
end loop
```

10.5 ε-Soft Monte Carlo Policy Iteration

Exploring starts are often infeasible in practice (we cannot teleport the agent to arbitrary states). The alternative is an **ε-soft policy**: assign positive probability to all actions at every step.

ε-Greedy Policy (the standard ε-soft choice)

$$\pi(a \mid s) = \begin{cases} 1 - \varepsilon + \dfrac{\varepsilon}{|\mathcal{A}(s)|} & \text{if } a = \arg\max_{a'} Q(s, a') \\ \dfrac{\varepsilon}{|\mathcal{A}(s)|} & \text{otherwise} \end{cases}$$

The greedy action receives most of the probability, but every action is explored with at least probability $\varepsilon/|\mathcal{A}(s)|$. This guarantees every (s, a) is visited infinitely often, so Q estimates converge. Even with forced exploration, the ε-greedy policy improvement theorem guarantees monotonic improvement toward the optimal ε-soft policy.

ε-Soft MC Policy Iteration

Input: $\varepsilon > 0$

```
Initialise π as ε-soft; Q(s, a) ∈ ℝ; Returns(s, a) ← ∅
loop for each episode:
    Generate episode following π: S_0, A_0, R_1, ..., S_T
    G ← 0
    for t = T − 1, ..., 0 do
        G ← γG + R_{t+1}
        if (S_t, A_t) first occurrence then
            Append G to Returns(S_t, A_t);   Q(S_t, A_t) ← average(...)
        end if
        A* ← arg max_a Q(S_t, a)
        for all a ∈ A(S_t) do
            π(a | S_t) ← { 1 − ε + ε/|A(S_t)|   a = A*
                         { ε/|A(S_t)|           otherwise
        end for
    end for
end loop
```

11 Off-Policy Learning

11.1 On-Policy vs. Off-Policy

In **on-policy** learning, the agent evaluates and improves the very policy it uses to generate data. For example, with ε-greedy MC, the agent follows an ε-greedy policy and learns the value of that same policy. The consequence is that it cannot learn a purely greedy optimal policy: the policy it learns must always include the exploratory noise.

In **off-policy** learning, the agent separates two policies: the **behaviour policy** b (used to collect data) and the **target policy** π (the policy being evaluated or optimised). Data is generated under b, but the agent uses it to learn about π. This decoupling allows learning an optimal deterministic target policy while the behaviour policy keeps exploring safely.

	Behaviour policy	**Learned policy**
On-policy	π	π
Off-policy	b	$\pi \neq b$

The **coverage condition** must hold: $\pi(a \mid s) > 0 \Rightarrow b(a \mid s) > 0$. Any action the target might take must also be possible under the behaviour policy; otherwise, we can never learn its value.

11.2 Importance Sampling

Since data comes from b but we want expectations under π, we correct the mismatch using **importance sampling**. For a random variable x: $\mathbb{E}_p[x] = \mathbb{E}_q[x\, p(x)/q(x)]$.

For a trajectory $\tau = (S_t, A_t, \ldots, S_T)$, the transition probabilities cancel in the ratio (they appear identically in numerator and denominator), leaving only policy probabilities:

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)}$$

This ratio reweights returns observed under b as if they had been observed under π.

Ordinary IS (unbiased, high variance): $V_\pi(s) \approx \frac{1}{N(s)} \sum_i \rho^{(i)} G^{(i)}$.

Weighted IS (biased, much lower variance — preferred in practice): $V_\pi(s) \approx \frac{\sum_i \rho^{(i)} G^{(i)}}{\sum_i \rho^{(i)}}$.

Ordinary IS can be highly unstable because importance ratios can be very large when π and b differ greatly. Weighted IS normalises by the sum of ratios, greatly reducing variance at the cost of a small bias that vanishes asymptotically.

12 Temporal-Difference Learning

12.1 Motivation

DP is model-based. MC is model-free but requires complete episodes. **Temporal-Difference** (TD) learning combines both advantages: it is model-free *and* it updates estimates after every single step, without waiting for the episode to end. This is achieved through **bootstrapping**: the current estimate $V(S_{t+1})$ is used as a proxy for the unknown future return.

12.2 TD(0): Core Idea and Update

Recall $V_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$. MC uses the full observed return G_t. TD replaces it with a *one-step approximation*:

$$G_t \approx R_{t+1} + \gamma V(S_{t+1}) \qquad \text{(TD target)}$$

TD(0) Update Rule

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{\big[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\big]}_{\delta_t = \text{TD error}}$$

$\delta_t > 0$ means the outcome was better than expected (increase $V(S_t)$); $\delta_t < 0$ means worse (decrease it). The update happens immediately after each step — no complete episode required.

TD(0) Policy Evaluation

Input: policy π, step size $\alpha \in (0, 1]$

```
Initialise V(s) arbitrarily; V(terminal) = 0
loop for each episode:
    Initialise S
    repeat
        Choose A ~ π(· | S); take A; observe R and S'
        V(S) ← V(S) + α[R + γV(S') − V(S)];   S ← S'
    until S is terminal
end loop
```

12.3 Random Walk Example

Consider the chain: L-term $\leftrightarrow A \leftrightarrow B \leftrightarrow C \leftrightarrow D \leftrightarrow E \leftrightarrow$ R-term. The agent starts at C, moves left or right equiprobably. Only reaching the right terminal gives reward $+1$; all other transitions give 0. With $\gamma = 1$, the true value $V(s)$ equals the probability of eventually reaching the right terminal ($V(A) = 1/6, V(B) = 2/6, \ldots, V(E) = 5/6$).

TD(0) propagates information *before the episode ends*: moving from C to D immediately updates $V(C)$ toward $V(D)$. This online propagation is the main advantage over MC, which would have to wait until the episode terminates (which could take many steps) to learn anything.

12.4 MC vs. TD: Bias-Variance Trade-off

	Monte Carlo	**TD(0)**
Target	Full return G_t	$R_{t+1} + \gamma V(S_{t+1})$
Bias	Unbiased	Biased (uses estimates)
Variance	High (many random steps)	Low (one step of randomness)
Updates	After complete episode	After every step
Convergence	Slower early on	Faster early on
Tasks	Episodic only	Episodic and continuing

The bias in TD comes from using the imperfect current estimate $V(S_{t+1})$; however, it introduces far less variance than using the full trajectory. In practice, TD often converges faster, especially in early training.

12.5 Bootstrapping and Sampling: Taxonomy

- **Bootstrapping**: update using existing estimates (DP and TD do this; MC does not).
- **Sampling**: use actual experience rather than full expectations (MC and TD do this; DP does not).

TD is unique in doing both, placing it between DP and MC.

13 SARSA and Q-Learning

13.1 From Evaluation to Control

For control (finding the optimal policy), TD must estimate $Q_\pi(s,a)$ rather than $V_\pi(s)$, since Q_π allows model-free policy improvement: $\pi'(s) = \arg\max_a Q_\pi(s,a)$. The idea is to replace the MC return with a one-step TD approximation: $G_t \approx R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$.

13.2 SARSA: On-Policy TD Control

SARSA Update Rule

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\big]$$

The name comes from the five-tuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$. A_{t+1} is chosen from the *same policy* being improved (e.g., ε-greedy). This makes SARSA **on-policy**: it evaluates and improves the policy it actually follows, including its exploratory behaviour.

SARSA

Input: $\alpha \in (0,1]$, $\varepsilon > 0$

```
Initialise Q(s,a) arbitrarily; Q(terminal, ·) = 0
loop for each episode:
    Initialise S; choose A from S via ε-greedy on Q
    repeat
        Take A; observe R and S'; choose A' from S' via ε-greedy on Q
        Q(S,A) ← Q(S,A) + α[R + γQ(S',A') − Q(S,A)]
        S ← S';   A ← A'
    until S is terminal
end loop
```

13.3 Q-Learning: Off-Policy TD Control

Q-learning modifies only the target: instead of using the action actually taken at S_{t+1}, it uses the *best* possible action:

Q-Learning Update Rule

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\big]$$

The target $R_{t+1} + \gamma \max_a Q(S_{t+1}, a)$ assumes the agent will act optimally from S_{t+1} onward, regardless of what it actually does. Q-learning directly approximates Q^*, making it **off-policy**:

the behaviour policy (e.g., ε-greedy for exploration) differs from the greedy target policy.

Q-Learning

Input: $\alpha \in (0, 1]$, $\varepsilon > 0$

```
Initialise Q(s, a) arbitrarily; Q(terminal, ·) = 0
loop for each episode:
    Initialise S
    repeat
        Choose A via ε-greedy on Q                    ▷ behaviour policy
        Take A; observe R and S'
        Q(S, A) ← Q(S, A) + α[R + γ max_a Q(S', a) − Q(S, A)]
        S ← S'
    until S is terminal
end loop
```

13.4 Bellman Equation Correspondence

SARSA vs. Q-Learning: Theoretical Grounding

- **SARSA** implements the Bellman *expectation* equation:

$$Q_\pi(s, a) = \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma \sum_{a'} \pi(a' \mid s')\, Q_\pi(s', a')\big]$$

  It evaluates and improves the current policy π (including its exploration).

- **Q-learning** implements the Bellman *optimality* equation:

$$Q^*(s, a) = \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma \max_{a'} Q^*(s', a')\big]$$

  It directly approximates Q^*, bypassing the current policy.

13.5 Cliff Walking: A Key Illustration

In the cliff-walking gridworld, the agent must travel from start to goal along a grid with a cliff at the bottom. Falling off the cliff gives reward -100 and resets to start.

The shortest path runs directly along the top of the cliff and is optimal in the limit but risky during learning: an ε-greedy agent will occasionally take random steps and fall. SARSA accounts for this exploratory behaviour in its update (it uses the action *actually taken* next), so it learns to stay away from the cliff edge to avoid accidental falls during training. Q-learning ignores the exploration noise in its update (it uses $\max_a Q$ instead), so it learns the shortest path close to the cliff, optimistic about future behaviour.

SARSA $\rightarrow$ safe path far from cliff $\rightarrow$ higher reward *during training*.
Q-learning $\rightarrow$ optimal path close to cliff $\rightarrow$ better *asymptotic* policy.

This trade-off is fundamental: SARSA performs better while the agent is still exploring; Q-learning converges to the globally better policy once $\varepsilon \rightarrow 0$.

14 n-Step Methods and Eligibility Traces

14.1 n-Step Returns

TD(0) uses one step of actual reward before bootstrapping; MC uses the complete episode. The natural generalisation is to use n steps of actual reward and then bootstrap:

n-Step Return

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

$n = 1$ recovers TD(0); $n \to \infty$ recovers Monte Carlo. Increasing n reduces bias (less reliance on potentially inaccurate estimates) but increases variance (more random steps are included). The optimal n is problem-dependent.

14.2 The λ-Return

Rather than choosing a single n, the **λ-return** combines *all* n-step returns with exponentially decaying weights:

λ-Return

$$G_t^{\lambda} = (1-\lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}, \qquad \lambda \in [0,1]$$

$\lambda = 0$: TD(0). $\lambda = 1$: Monte Carlo. Intermediate λ provides a smooth interpolation with a single continuous parameter controlling the bias-variance trade-off.

14.3 Eligibility Traces: Efficient Implementation of TD(λ)

Computing all n-step returns and weighting them is computationally expensive. **Eligibility traces** achieve the equivalent result online, without storing the trajectory.

The intuition: when a surprising TD error δ_t occurs at time t, it should update not only $V(S_t)$ but also the values of recently visited states (they "deserve credit" for the outcome). The eligibility trace $e_t(s)$ tracks how recently and how frequently state s was visited.

Eligibility Trace and TD(λ) Update

$$e_t(s) = \gamma\lambda\, e_{t-1}(s) + \mathbf{1}(s = S_t)$$

At each step, the trace of S_t is incremented by 1; all other traces decay by $\gamma\lambda$. Updates are then proportional to the trace:

$$V(s) \leftarrow V(s) + \alpha\, \delta_t\, e_t(s) \quad \forall s, \qquad \delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

States visited very recently have large traces and receive large updates. States visited several steps ago have smaller traces and receive proportionally smaller updates. This implements **backward credit assignment**: a surprising outcome propagates updates efficiently through the history of recently visited states, all in a single forward pass.

15 Function Approximation

15.1 Why Function Approximation?

All methods so far represent the value function as a lookup table: one entry per state (or state-action pair). This is only feasible when $|\mathcal{S}|$ is small. In most real problems — robot locomotion, video games, financial trading — the state space is enormous or even continuous, making tabular representation impossible. **Function approximation** (FA) addresses this by representing value functions with a parameterised family, such as a linear model or a neural network, that can generalise across states.

Instead of a table $V(s)$, we have a parameterised function $\hat{V}(s;\mathbf{w})$ where $\mathbf{w} \in \mathbb{R}^d$ is a weight vector with $d \ll |\mathcal{S}|$. Similarly for the action-value function: $\hat{Q}(s,a;\mathbf{w})$. The goal is to find $\mathbf{w}$ such that $\hat{V}(s;\mathbf{w}) \approx V_\pi(s)$ for all s.

15.2 Linear Function Approximation

The simplest approximator is a linear combination of **features**:

$$\hat{V}(s;\mathbf{w}) = \mathbf{w}^\top \mathbf{x}(s) = \sum_{i=1}^{d} w_i \, x_i(s)$$

where $\mathbf{x}(s) = (x_1(s), \ldots, x_d(s))^\top$ is a feature vector representing state s. The weight vector $\mathbf{w}$ is learned; the features are fixed and encode domain knowledge (e.g., position, velocity, distance to goal).

The quality of the approximation is measured by the **Mean Squared Value Error**:

$$\overline{VE}(\mathbf{w}) = \sum_{s \in \mathcal{S}} \mu(s) \big[V_\pi(s) - \hat{V}(s;\mathbf{w})\big]^2$$

where $\mu(s)$ is a weighting that reflects how often state s is visited (on-policy distribution). We minimise this via **stochastic gradient descent** (SGD): at each step, update $\mathbf{w}$ in the direction that reduces the error for the current state.

15.3 Semi-Gradient TD(0)

For tabular TD, the update was $V(S_t) \leftarrow V(S_t) + \alpha\delta_t$. With function approximation, the analogous update adjusts $\mathbf{w}$ using the gradient of $\hat{V}$:

Semi-Gradient TD(0) Update

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \, \delta_t \, \nabla_{\mathbf{w}} \hat{V}(S_t;\mathbf{w})$$

where $\delta_t = R_{t+1} + \gamma \hat{V}(S_{t+1};\mathbf{w}) - \hat{V}(S_t;\mathbf{w})$ is the TD error.

The update is called "semi-gradient" because it treats the TD target $R_{t+1} + \gamma\hat{V}(S_{t+1};\mathbf{w})$ as a fixed constant when computing the gradient — the gradient of the target w.r.t. $\mathbf{w}$ is ignored. This makes the update computationally simple but introduces a bias. For linear FA, semi-gradient TD(0) converges to a solution near the true optimum under on-policy conditions.

For linear FA: $\nabla_{\mathbf{w}} \hat{V}(s;\mathbf{w}) = \mathbf{x}(s)$, so the update simplifies to: $\mathbf{w} \leftarrow \mathbf{w} + \alpha \, \delta_t \, \mathbf{x}(S_t)$.

15.4 Non-Linear Function Approximation and Neural Networks

When the value function has complex structure, linear models are not expressive enough. Neural networks can approximate arbitrary functions and learn useful representations automatically from raw inputs.

A neural network represents $\hat{V}(s;\mathbf{w})$ (or $\hat{Q}(s,a;\mathbf{w})$) as a composition of nonlinear transformations:

$$\hat{V}(s;\mathbf{w}) = f_L(\cdots f_2(f_1(s;\mathbf{w}_1);\mathbf{w}_2)\cdots;\mathbf{w}_L)$$

where f_ℓ is the ℓ-th layer (e.g., a linear transformation followed by a ReLU non-linearity) and $\mathbf{w} = (\mathbf{w}_1, \ldots, \mathbf{w}_L)$ collects all network parameters.

Training uses **backpropagation** to compute the gradient $\nabla_{\mathbf{w}}\hat{V}(s;\mathbf{w})$ efficiently, and then applies the semi-gradient update. The challenge is that RL training is inherently non-stationary (the target changes as $\mathbf{w}$ changes) and the data is correlated (consecutive transitions are similar), both of which can destabilise gradient descent.

Challenges of Function Approximation in RL

- **Moving target:** the TD target depends on $\mathbf{w}$ itself, creating a feedback loop that can cause divergence.
- **Correlated data:** consecutive transitions are highly correlated, violating the i.i.d. assumption of SGD.
- **Deadly triad:** the combination of function approximation + bootstrapping + off-policy training can cause divergence. Solutions include experience replay and target networks (see Deep RL).

16 Deep Reinforcement Learning

16.1 Motivation

Deep RL combines deep neural networks with RL algorithms to handle high-dimensional, raw sensory inputs (e.g., pixels from a video game screen). The breakthrough came with the **Deep Q-Network** (DQN), which showed for the first time that a single neural network trained with RL could achieve human-level performance across a wide range of Atari games, learning directly from raw pixel inputs without any game-specific feature engineering.

16.2 Deep Q-Network (DQN)

DQN uses a neural network $\hat{Q}(s,a;\mathbf{w})$ to approximate Q^* and applies Q-learning updates. Two key innovations address the instability problems of naive neural network Q-learning:

Experience Replay

Instead of updating $\mathbf{w}$ immediately after each transition, transitions $(S_t, A_t, R_{t+1}, S_{t+1})$ are stored in a large **replay buffer** $\mathcal{D}$. At each update step, a random mini-batch of transitions is sampled from $\mathcal{D}$:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha\, \nabla_{\mathbf{w}} \sum_{(s,a,r,s')\in\text{batch}} \left[r + \gamma \max_{a'} \hat{Q}(s',a';\mathbf{w}^-) - \hat{Q}(s,a;\mathbf{w}) \right]^2$$

Benefits: (1) breaks correlations between consecutive transitions (data looks more i.i.d.); (2) each transition can be reused multiple times, improving sample efficiency.

Target Network

A separate **target network** $\hat{Q}(s, a; \mathbf{w}^-)$ is used to compute the TD target. The target network parameters $\mathbf{w}^-$ are a copy of $\mathbf{w}$ that is updated only periodically (every C steps), rather than at every gradient step.

Benefit: the target $r + \gamma \max_{a'} \hat{Q}(s', a'; \mathbf{w}^-)$ is fixed over many update steps, breaking the harmful feedback loop where $\mathbf{w}$ chases a constantly moving target.

DQN Algorithm

```
Initialise replay buffer D; initialise Q̂(·; w) with random weights
Initialise target network: w⁻ ← w
loop for each episode:
    Initialise state S (e.g., stack of last 4 frames)
    repeat
        Choose A via ε-greedy w.r.t. Q̂(S, ·; w)
        Take A; observe R and S′; store (S, A, R, S′) in D
        Sample random mini-batch {(s_j, a_j, r_j, s′_j)} from D
        Set targets: y_j = r_j + γ max_a′ Q̂(s′_j, a′; w⁻)
        Update w by minimising Σ_j (y_j − Q̂(s_j, a_j; w))²
        Every C steps: w⁻ ← w
        S ← S′
    until S is terminal
end loop
```

16.3 DQN in Practice: Atari Games

In the original DQN paper (Mnih et al., 2015), the input to the network is a stack of the last 4 greyscale frames (to capture motion), and the output is $\hat{Q}(s, a; \mathbf{w})$ for all actions a simultaneously (one output neuron per action). The network uses convolutional layers to process pixel inputs, followed by fully connected layers. A single fixed architecture and set of hyperparameters was used across all games.

DQN was able to match or surpass human performance on many Atari games, demonstrating that end-to-end deep RL from raw pixels is feasible. The key insight is that deep networks can learn useful feature representations automatically, eliminating the need for hand-crafted features.

16.4 Limitations of DQN and Extensions

DQN has several known limitations:

- **Overestimation bias:** $\max_{a'} \hat{Q}(s', a'; \mathbf{w}^-)$ tends to overestimate the true value because the same weights are used to both select and evaluate the best action. **Double DQN** fixes this by using the online network to select the action and the target network to evaluate it.
- **Uniform experience replay:** DQN samples transitions uniformly from the replay buffer. **Prioritised Experience Replay** samples more frequently from transitions with large TD errors, improving learning efficiency.
- **Discrete actions only:** DQN outputs one Q-value per action and requires enumeration, so it cannot handle continuous action spaces. Policy gradient methods (see next section) address this.

17 Policy Gradient Methods

17.1 Why Policy Gradients?

All methods so far (DP, MC, TD, DQN) work by learning a value function and then deriving a policy from it (e.g., by acting greedily). This is the *indirect* approach to policy optimisation. **Policy gradient** methods take a fundamentally different, *direct* approach: they represent the policy explicitly as a parameterised function $\pi(a \mid s; \boldsymbol{\theta})$ and directly optimise the policy parameters $\boldsymbol{\theta}$ to maximise expected return.

This direct approach has several advantages over value-based methods:

- **Continuous action spaces:** policy gradients work naturally when $\mathcal{A}$ is continuous (e.g., robot joint angles), whereas Q-learning requires $\max_a Q(s, a)$ which is intractable for continuous $\mathcal{A}$.
- **Stochastic policies:** the policy is stochastic by construction. This is useful in partially observable environments and in multi-agent games where a deterministic policy can be exploited.
- **Smooth optimisation:** small changes in $\boldsymbol{\theta}$ produce small changes in π, making gradient-based optimisation well-behaved.

17.2 The Policy Gradient Objective

Define the performance objective as the expected return under policy $\pi(\cdot; \boldsymbol{\theta})$:

$$J(\boldsymbol{\theta}) = \mathbb{E}_{\tau \sim \pi_{\boldsymbol{\theta}}}\left[G_0\right] = \mathbb{E}_{\tau \sim \pi_{\boldsymbol{\theta}}}\left[\sum_{t=0}^{T} \gamma^t R_{t+1}\right]$$

We want to find $\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$ by gradient ascent: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$.

The challenge is computing $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$: the expectation is taken over trajectories whose distribution depends on $\boldsymbol{\theta}$, making direct differentiation difficult.

17.3 The Policy Gradient Theorem

Policy Gradient Theorem

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}}\left[\sum_{t=0}^{T} \nabla_{\boldsymbol{\theta}} \log \pi(A_t \mid S_t; \boldsymbol{\theta}) \cdot G_t\right]$$

This remarkable identity says: the gradient of the expected return equals the expected value of the score function $\nabla_{\boldsymbol{\theta}} \log \pi$ weighted by the return G_t.

Intuition: the term $\nabla_{\boldsymbol{\theta}} \log \pi(A_t \mid S_t; \boldsymbol{\theta})$ points in the direction that increases the probability of action A_t in state S_t. Multiplying by G_t means we push the policy *more strongly* in that direction when the return was large (the action was good) and *less strongly* (or in the opposite direction) when the return was small.

Why $\log \pi$? The log-derivative trick: $\nabla_\theta \pi = \pi \cdot \nabla_\theta \log \pi$, which allows us to write the gradient as an expectation under π without differentiating through the sampling process.

17.4 REINFORCE: Monte Carlo Policy Gradient

REINFORCE is the simplest policy gradient algorithm. It estimates $\nabla_{\boldsymbol{\theta}} J$ using full MC returns from sampled episodes:

REINFORCE (Monte Carlo Policy Gradient)

Input: differentiable policy $\pi(a \mid s; \boldsymbol{\theta})$, step size $\alpha > 0$

```
Initialise θ arbitrarily
loop for each episode:
    Generate episode S_0, A_0, R_1, ..., S_T following π(·; θ)
    for t = 0, 1, ..., T do
        Compute return: G_t = Σ_{k=t}^{T-1} γ^{k-t} R_{k+1}
        Update: θ ← θ + α γ^t G_t ∇_θ log π(A_t | S_t; θ)
    end for
end loop
```

Properties: REINFORCE produces an *unbiased* estimate of the policy gradient, but it has *very high variance* because G_t depends on the full trajectory, which is highly variable. This makes learning slow and unstable.

17.5 Variance Reduction: The Baseline

A key variance reduction technique is to subtract a **baseline** $b(s)$ from the return:

$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \mathbb{E}_{\pi_{\boldsymbol{\theta}}}\big[\nabla_{\boldsymbol{\theta}} \log \pi(A_t \mid S_t; \boldsymbol{\theta}) \cdot \big(G_t - b(S_t)\big)\big]$$

This is still an unbiased estimator of the gradient (the baseline does not introduce bias, since $\mathbb{E}[\nabla_\theta \log \pi \cdot b(S_t)] = 0$), but it can dramatically reduce variance by centring the return. The natural choice is $b(S_t) = V_\pi(S_t)$: then $G_t - V_\pi(S_t)$ is the **advantage function** $A_\pi(S_t, A_t)$, measuring how much better action A_t is than the average action in state S_t.

17.6 Actor-Critic Methods

REINFORCE uses MC returns for G_t, which requires complete episodes and has high variance. **Actor-Critic** methods reduce variance further by replacing G_t with a TD estimate, bootstrapping off a learned value function (*critic*).

The architecture has two components:

- **Actor:** the policy $\pi(a \mid s; \boldsymbol{\theta})$, updated via policy gradient.
- **Critic:** the value function $\hat{V}(s; \mathbf{w})$, updated via TD learning.

Actor-Critic Update (one-step TD)

At each step:

$$\delta_t = R_{t+1} + \gamma \hat{V}(S_{t+1}; \mathbf{w}) - \hat{V}(S_t; \mathbf{w}) \quad \text{(TD error = advantage estimate)}$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha_c \, \delta_t \, \nabla_{\mathbf{w}} \hat{V}(S_t; \mathbf{w}) \quad \text{(critic update)}$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha_a \, \delta_t \, \nabla_{\boldsymbol{\theta}} \log \pi(A_t \mid S_t; \boldsymbol{\theta}) \quad \text{(actor update)}$$

The TD error δ_t serves as an estimate of the advantage: it is positive when the outcome was better than the critic expected, and negative when it was worse.

One-Step Actor-Critic

Input: $\alpha_a, \alpha_c > 0$

```
Initialise θ, w arbitrarily
loop for each episode:
    Initialise S;   I ← 1
    repeat
        Choose A ~ π(· | S; θ);   take A; observe R and S'
        δ ← R + γV̂(S'; w) − V̂(S; w)
        w ← w + α_c δ ∇_w V̂(S; w)
        θ ← θ + α_a I δ ∇_θ log π(A | S; θ)
        I ← γI;   S ← S'
    until S is terminal
end loop
```

17.7 Comparing Value-Based and Policy-Based Methods

	Value-based (e.g. DQN)	**Policy-based** (e.g. REINFORCE)
What is learned	$Q^*(s, a; \mathbf{w})$	$\pi(a \mid s; \boldsymbol{\theta})$
Policy	Implicit (greedy w.r.t. Q)	Explicit
Action spaces	Discrete (typically)	Discrete or continuous
Stochastic policies	Indirect only	Natural
Variance	Lower	Higher (without baseline)
Bias	Higher (bootstrapping)	Lower (MC)

Actor-critic methods combine both: explicit policy (actor) + learned value function (critic), getting lower variance than REINFORCE and handling continuous actions unlike DQN.

18 Big Picture Summary

Method	**Model?**	**Bootstrap?**	**Sample?**	**FA?**	**Policy type**
DP	Required	Yes	No	No	Implicit
MC	No	No	Yes	No	Implicit
TD(0)/SARSA	No	Yes	Yes	No	Implicit
Q-Learning	No	Yes	Yes	No	Implicit
Semi-grad TD	No	Yes	Yes	Yes	Implicit
DQN	No	Yes	Yes	Yes (deep)	Implicit
REINFORCE	No	No	Yes	Yes	Explicit
Actor-Critic	No	Yes	Yes	Yes	Explicit

Generalised Policy Iteration: The Unifying Framework

Almost every RL method can be understood as an instance of **Generalised Policy Iteration** (GPI): two interacting processes — evaluation and improvement — that push the system toward

optimality.

$$\underbrace{V \approx V_\pi}_{\text{Evaluation}} \qquad \longleftrightarrow \qquad \underbrace{\pi \approx \text{greedy}(V)}_{\text{Improvement}}$$

In DP: both steps are complete and alternating. In MC/TD: interleaved at varying granularities. In policy gradient: evaluation is done by the critic; improvement is done directly via gradient ascent on $J(\boldsymbol{\theta})$. In all cases, the interplay between evaluation and improvement drives the system toward the optimal policy.